# Implementação de Algoritmos para Resolução do Problema das 8 Rainhas

**Caetano A. de Matos**[1], **Ruan Vasconcelos**[1]

[1]Instituto Federal de Educação, Ciência e Tecnologia do Rio Grande do Sul
Campus Ibirubá
Rua Nelsi Ribas Fritsch, 1111 – CEP: 98200-000 – Ibirubá – RS – Brasil

***Abstract.*** *This article presents the implementation and comparative analysis of three search and optimization algorithms — Hill Climbing with Random Restart, Simulated Annealing, and Genetic Algorithm — applied to solving the classic 8 Queens problem. The objective is to place eight queens on an* $8 \times 8$ *chessboard such that no two queens threaten each other. The work describes the computational modeling used, highlighting the representation of the board in a one-dimensional array to optimize the search space, and explores the impact of parameter configurations on the performance of each method. Empirical tests demonstrated that, although all approaches successfully find a valid state, local search methods are significantly more efficient. Hill Climbing with Random Restart stood out for its extreme speed and computational simplicity, establishing itself as the most pragmatic approach for this specific scenario.*

***Resumo.*** *Este artigo apresenta a implementação e análise comparativa de três algoritmos de busca e otimização — Subida de Encosta com Reinício Aleatório, Têmpera Simulada e Algoritmo Genético — aplicados à resolução do clássico problema das 8 rainhas. O objetivo é posicionar oito rainhas em um tabuleiro de xadrez* $8 \times 8$ *sem que haja conflitos entre elas. O trabalho descreve a modelagem computacional utilizada, destacando a representação do tabuleiro em um vetor unidimensional para otimização do espaço de busca, e explora o impacto das configurações de parâmetros no desempenho de cada método. Os testes empíricos demonstraram que, embora todas as abordagens garantam o sucesso na resolução, os métodos de busca local são significativamente mais eficientes. A Subida de Encosta com Reinício Aleatório destacou-se pela extrema rapidez e simplicidade computacional, consolidando-se como a abordagem mais pragmática para este cenário específico.*

## 1. Introdução

Este artigo apresenta o desenvolvimento e a análise de um sistema computacional voltado para a resolução do clássico problema das 8 rainhas. O desafio consiste em encontrar uma disposição para oito rainhas em um tabuleiro de xadrez $8 \times 8$ de forma que nenhuma delas esteja sob ataque mútuo, respeitando estritamente as restrições de linhas, colunas e diagonais. O código-fonte completo do projeto pode ser acessado em <https://github.com/RuanVasco/ai-projects/tree/main/8-queens-solver>.

Para solucionar este problema de otimização combinatória, foram implementadas e avaliadas três abordagens algorítmicas distintas: Subida de Encosta (*Hill Climbing*) acoplada ao mecanismo de Reinício Aleatório, Têmpera Simulada (*Simulated Annealing*) e

um Algoritmo Genético. O trabalho detalha a modelagem matemática adotada — especificamente a redução do tabuleiro a vetores unidimensionais para ganho de performance — e o impacto do ajuste fino de parâmetros no comportamento de exploração e convergência de cada método.

Além da análise teórica e da avaliação empírica de desempenho temporal, o projeto conta com uma interface desenvolvida para a configuração dinâmica dos parâmetros e a visualização comparativa dos resultados. Este escopo constitui um estudo prático e direto sobre as vantagens e desvantagens da aplicação de heurísticas locais e metaheurísticas populacionais no campo da Inteligência Artificial.

## 2. Desenvolvimento

O problema das 8 rainhas consiste em posicionar oito rainhas em um tabuleiro de xadrez $8 \times 8$ de modo que nenhuma delas ataque outra (mesma linha, coluna ou diagonal). O espaço de busca para este problema, considerando a restrição de uma rainha por coluna, é de $8^8$ combinações. Para solucioná-lo, foram implementados três algoritmos de busca e otimização, cujas abordagens e resultados são detalhados a seguir.

A Figura 1 apresenta a interface desenvolvida para a configuração dos parâmetros iniciais de cada algoritmo, permitindo o ajuste fino para o balanceamento entre exploração e explotação do espaço de busca.

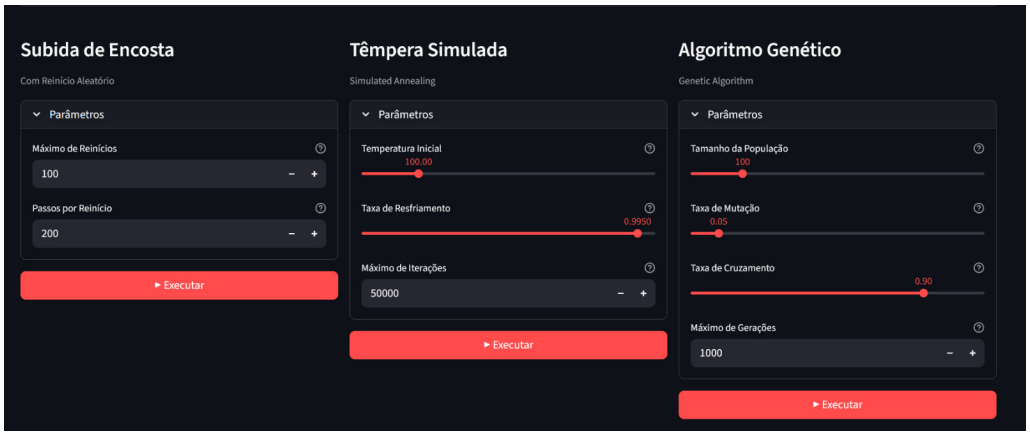


**Figura 1. Interface de configuração de parâmetros para Subida de Encosta, Têmpera Simulada e Algoritmo Genético.**

### 2.1. Subida de Encosta (Hill Climbing)

Este algoritmo é uma heurística de busca local de natureza estritamente gulosa. A partir de um estado de tabuleiro atual, o algoritmo gera sua vizinhança completa — no caso das 8 rainhas, isso é feito movendo uma única rainha para qualquer outra linha de sua respectiva coluna (totalizando 56 vizinhos possíveis a cada passo). O algoritmo avalia a função de custo (número de conflitos) de todos esses vizinhos e transita iterativamente para o estado que apresenta a maior redução no número de ataques.

A principal limitação dessa abordagem matemática é a forte tendência de convergência para "ótimos locais", ou seja, configurações onde os conflitos não são zero, mas nenhum movimento individual e isolado é capaz de melhorar a situação. Para contornar

essa deficiência, acopla-se a técnica de **Reinício Aleatório**. Com ela, ao detectar que o algoritmo ficou preso em um ótimo local ou platô, a busca é interrompida, o tabuleiro atual é descartado, e o processo recomeça a partir de um novo estado gerado de forma totalmente estocástica, repetindo-se até que o ótimo global (zero conflitos) seja alcançado.

- **Configuração:** Máximo de 100 reinícios e 200 passos por reinício.
- **Justificativa:** O comportamento de "serra"no gráfico de convergência indica que a estagnação é comum, mas o reinício rápido permite encontrar a solução global com baixo custo computacional.

### 2.2. Têmpera Simulada (Simulated Annealing)

Inspirada no processo termodinâmico de recozimento físico de metais — onde o material é aquecido a altas temperaturas e resfriado de forma controlada para alcançar uma estrutura cristalina de energia mínima e sem falhas —, esta meta-heurística estende a busca local ao permitir transições ocasionais para estados piores.

A cada iteração, um vizinho aleatório é selecionado. Se o vizinho possuir menos conflitos ($\Delta < 0$), a transição ocorre imediatamente. Caso possua mais conflitos ($\Delta > 0$), o movimento ainda pode ser aceito segundo a probabilidade de Boltzmann, denotada por $P = e^{-\Delta/T}$, onde $\Delta$ é a variação de custo e $T$ é a temperatura atual do sistema. Nas iterações iniciais, com a temperatura $T$ elevada, a probabilidade de aceitar movimentos de piora é alta, garantindo uma ampla exploração do espaço de busca e a fuga de ótimos locais. Conforme as iterações progridem e a temperatura decai, a probabilidade $P$ tende a zero, e o algoritmo assume um comportamento progressivamente mais guloso (focando na explotação refinada da região em que se encontra).

- **Configuração:** Temperatura inicial 100.0 e taxa de resfriamento de 0.995.
- **Justificativa:** A taxa de 0.995 garante um resfriamento lento o suficiente para que o algoritmo explore o espaço antes de convergir para a solução final quando a temperatura se aproxima de zero.

### 2.3. Algoritmo Genético

Baseado nos princípios da teoria da evolução e genética biológica, este método de busca global paralela não opera sobre um único estado por vez, mas sim sobre uma ampla população de soluções candidatas. No contexto do problema, cada indivíduo é representado por um cromossomo (um vetor numérico indicando a linha de cada rainha). A cada geração, a qualidade das soluções é quantificada por uma função de *fitness* (aptidão), calculada inversamente ao número de conflitos do tabuleiro.

A evolução iterativa da população ocorre através da aplicação de três operadores estocásticos: a **Seleção**, que privilegia os indivíduos com maior *fitness* para passarem seus genes adiante (utilizando métodos como a seleção por torneio); o **Cruzamento** (*Crossover*), que recombina os segmentos numéricos de dois vetores "pais"para gerar novos tabuleiros descendentes; e a **Mutação**, que altera aleatoriamente a posição de uma rainha em uma taxa baixa controlada. A mutação é vital para reintroduzir diversidade genética no ecossistema artificial e prevenir a convergência prematura da população inteira para um único ótimo local.

- **Configuração:** População de 100 indivíduos, 90% de cruzamento e 5% de mutação.
- **Justificativa:** Esses valores garantem uma recombinação eficiente de boas características (fitness) e evitam a convergência prematura da população.

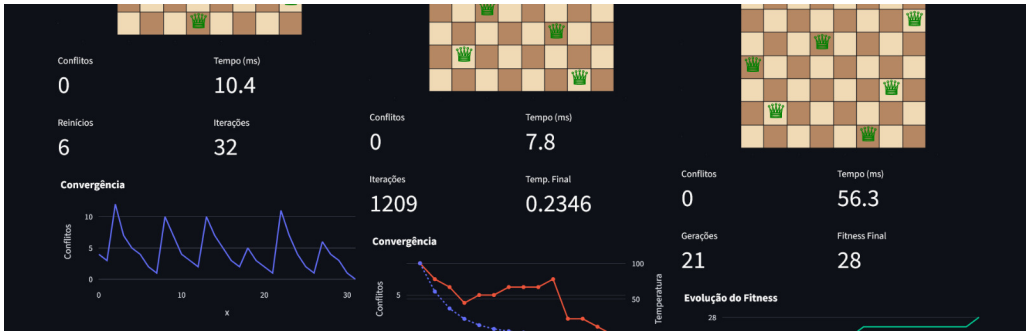


**Figura 2. Visualização das soluções encontradas e gráficos de convergência/fitness para cada método.**

## 2.4. Resultados e Discussão

A Figura 2 demonstra que todos os métodos atingiram o estado de zero conflitos. Entretanto, a Figura 3 e a tabela a seguir revelam diferenças drásticas de desempenho.

**Figura 3. Comparação de tempo de execução e qualidade da solução entre os algoritmos.**

**Tabela 1. Métricas de desempenho coletadas nos testes.**

| Algoritmo | Conflitos | Tempo (ms) | Iterações/Gerações |
|---|---|---|---|
| Têmpera Simulada | 0 | 7.81 | 1209 |
| Subida de Encosta | 0 | 10.43 | 32 |
| Algoritmo Genético | 0 | 56.26 | 21 |

## 2.5. Análise Comparativa: Vantagens e Desvantagens

A análise empírica dos resultados evidenciou discrepâncias significativas relacionadas à eficiência algorítmica e ao custo computacional. A seguir, delineiam-se as vantagens e desvantagens de cada abordagem para o problema das 8 rainhas:

### 1. Subida de Encosta com Reinício Aleatório

- **Vantagens:** Destaca-se por ser extremamente leve em termos de alocação de memória, visto que mantém e avalia apenas um único estado ativo por vez. Apresenta

convergência agressiva e altíssima velocidade (10.43 ms). O mecanismo de reinício resolve a vulnerabilidade a ótimos locais de maneira simples e muito robusta para espaços de busca de pequeno a médio porte.
- **Desvantagens:** É um algoritmo "míope"e desprovido de memória sistêmica. Todo o progresso feito antes de um reinício é sumariamente descartado, tornando a busca global altamente dependente da estocasticidade. Em problemas de dimensionalidade muito maior, os reinícios passariam a consumir tempo excessivo.

**2. Têmpera Simulada**

- **Vantagens:** Obteve o melhor desempenho temporal absoluto (7.81 ms) nos testes. Diferente da Subida de Encosta, oferece um mecanismo matemático rigoroso para transitar pelas depressões do espaço de busca sem a necessidade de descartar e reiniciar o progresso alcançado. Mantém a vantagem da busca local quanto ao baixo uso de memória.
- **Desvantagens:** Sua eficácia é criticamente sensível à calibração de hiperparâmetros. Um ajuste inadequado da temperatura inicial ou da taxa de resfriamento pode arruinar a busca: se a temperatura cair muito devagar, degenera para uma busca aleatória ineficiente; se cair muito rápido, congela precocemente em ótimos locais.

**3. Algoritmo Genético**

- **Vantagens:** Possui notável poder de exploração paralela, avaliando múltiplas regiões do espaço de busca simultaneamente. Graças à dinâmica populacional e à mutação, é a técnica mais resiliente contra ótimos locais, dependendo consideravelmente menos da qualidade estocástica de uma única configuração inicial.
- **Desvantagens:** Apresentou o pior tempo de execução (56.26 ms), sendo quase 7 vezes mais lento que a Têmpera Simulada. Esse atraso deve-se ao elevado *overhead* computacional: o sistema despende tempo substancial gerenciando listas, instanciando novos vetores cromossômicos, calculando funções de custo repetidas vezes e aplicando os operadores biológicos. Trata-se de uma ferramenta superdimensionada (*overkill*) para a escala do problema das 8 rainhas.

## 3. Conclusão

Para o problema das 8 rainhas, os métodos de busca local (Subida de Encosta e Têmpera Simulada) mostraram-se superiores em eficiência. Conclui-se que a **Subida de Encosta com Reinício Aleatório** é a escolha mais equilibrada: é simples de implementar, exige o mínimo de memória e alcança a solução global em tempo quase instantâneo, superando a complexidade desnecessária do Algoritmo Genético para este cenário específico.

## Referências